Agahi: Development Plan

Team build plan for Build4Better (Codesphere 3.0). Build window: 9:00 AM to 5:00 PM PKT, Saturday 31 October 2026. Final submission on Hackathon Hub by 4:30 PM PKT (safety margin).

1. Goal and non-negotiables

Agahi helps older adults understand their bills and learn everyday WhatsApp and email tasks, step by step. It explains and teaches. It never performs tasks for the user. The plan below is built to finish on time with a working demo, even if the live AI service is slow or down during judging.

- **Working demo first.** Every judged feature must work with cached data, so the demo never depends on a live network call.
- **Scope is locked at 1:30 PM.** No new features after that point. Only bug fixes, copy, and polish.
- **Privacy and safety are never cut.** Privacy screen, AI estimate note, masked account numbers, and mock disclaimers are required.
- **Commit at least every 45 minutes** so the repository shows steady, original development history.

2. Scope lock

Priority	Feature	Done when
MUST	F1. Bill explainer: upload or pick a sample bill, see fields with plain-language explanations	Three sample bills always render correct cards; unknown fields show 'Not found on this bill'
MUST	F2. Glossary: 15 terms, tap to open a bottom sheet (English, with short Urdu where possible)	Tapping a highlighted term opens its definition offline
MUST	F3. WhatsApp guides: 4 guides (video call, voice message, share location, send a file or photo)	Each guide has 4 to 7 steps with Next, Back, and a 'Step X of Y' indicator; no AI call needed
MUST	F4. Gmail attachment guide, using mock screens	Steps cover compose, paperclip, choose file, confirm attachment, send
MUST	F5. Privacy screen, reachable in 2 taps from Home	States what is stored, what is not, and that AI can make mistakes
MUST	F6. Home screen with large tiles, plus text size and high-contrast settings	Buttons are at least 48 px tall; body text at least 18 px; toggles persist on reload
SHOULD	F7. Screen helper: upload a screenshot, get 3 to 5 steps	Works with 2 cached screenshots; live AI is a bonus, not a requirement
SHOULD	F8. Urdu labels on Home and guide titles	Urdu strings come from one JSON file; English shown if Urdu is missing
COULD	F9. Live bill extraction with Claude (live mode)	Works on sample bills; a toggle switches between live and cached
CUT	Voice agent, hosted voice agent, chatbot, accounts, database, analytics	Explicitly listed as out of scope in the README

Note on F9: live extraction is valuable for the Execution score, but it is the one feature with real risk. Build it last, after F1 to F6 pass the checklist. If it is not stable by 3:30 PM, ship with cached mode and say so in the README and video.

3. Tech stack

Layer	Choice	Reason
Frontend	React with Vite, plain CSS with CSS variables	Fast setup; CSS variables make text size and contrast toggles simple
Routing	React Router	Simple screen-to-screen navigation
Settings state	React Context, saved to localStorage	Keeps language, text size and contrast between visits
Backend	Node.js with Express (one server)	Keeps the API key off the client; few routes
AI (live mode only)	Claude API, model string claude-sonnet-5-5	Handles images, PDFs and screenshots in one model
File handling	Multer for uploads, Sharp to resize images, pdf-parse for text PDFs	Keeps requests small and fast
Icons	Lucide React	Clear, free, large-friendly
Fonts	Noto Sans, Noto Nastaliq Urdu (Google Fonts)	Reliable Urdu rendering
Testing	Manual checklist (Section 9) plus a few Vitest unit tests for the JSON parsers	Light but enough for a hackathon
Hosting (optional)	Frontend and backend on Render or Vercel free tiers	Only if time allows; a local demo is acceptable
Version control	GitHub, branches main and dev	Required by the rulebook; shows commit history

4. Data plan

Agahi stores almost nothing. That is deliberate and supports the Trust, Safety and Privacy score.

Data	Source	Where it lives
Uploaded bills and screenshots	User upload	Server memory during the request only; discarded after response; never written to disk
Sample bills (3)	Team-written fake data	client/src/data/samples/*.json
Cached extraction results	Generated once from the sample bills	server/cache/sample-results.json
Glossary (15 terms)	Team-written	client/src/data/glossary.json
Guides (WhatsApp x4, Gmail x1)	Team-written	client/src/data/guides/*.json
Screen helper cache (2 screenshots)	Team-written expected outputs	server/cache/screen-results.json
Mock screen images	Recreated by the team (Figma or Canva)	client/public/mock-screens/
Settings (language, text size, contrast)	User choice	Browser localStorage

Not collected: accounts, contacts, passwords, OTPs, PINs, analytics, and server logs of uploaded content. Turn off request body logging on the server.

Account numbers are masked to the last 4 digits in both the AI prompt and the UI. Sample bills already use masked numbers.

5. AI design (kept small)

- **POST /api/extract-bill** (live mode). Input: JPG, PNG or PDF, max 5 MB. Output: JSON with provider, amount_due, due_date, billing_period, account_number, previous_balance, late_fee, plus a one-line explanation per field.
- **System prompt rules:** return only JSON; no text outside the JSON; if a value is not clearly visible, return null; never guess; account number as last 4 digits only; never give payment advice.
- **Parsing:** the server validates the JSON. If parsing fails, it returns error code 'unreadable', and the UI shows a friendly retry message. It never shows raw model text.
- **POST /api/explain-screen** (screen helper). Returns app_name, description, steps (3 to 5), and confidence. If unsure, it says so and suggests asking a family member. It must warn the user never to share a password, OTP or PIN shown on screen.
- **Output rendering:** plain text only, never raw HTML.
- **Cached mode:** a toggle or environment flag (USE_CACHE=true) makes both endpoints return the stored results. Cached mode is the default for the final demo recording.

6. API contract (agree in the first 15 minutes)

Method and route	Request	Response
GET /api/samples	none	Array of 3 sample bill objects
POST /api/extract-bill	multipart, field 'file' (or sample_id in cached mode)	200: fields + explanations; 422: { error: 'unreadable' }
POST /api/explain-screen	multipart, field 'file' (or sample_id in cached mode)	200: { app_name, description, steps, confidence }; 422: { error: 'unreadable' }
GET /api/health	none	200: { status: 'ok', mode: 'cached' | 'live' }

7. Work split (two people)

Split by layer so neither person blocks the other. Person A owns the screens and content. Person B owns the server, AI and data. Both agree on the API contract before writing code.

Person A: Frontend and content	Person B: Backend, AI and data
React shell, routing, Home, Settings, Privacy screens Bill upload UI, result cards, glossary bottom sheet Guide list and step view components WhatsApp and Gmail guide JSON content and mock screen images Text size, contrast, Urdu fonts, button sizing Demo recording (with Person B)	Express server with the three routes plus health check Sample bills JSON and glossary JSON (English and Urdu) Cached results files for bills and screenshots Live mode: Claude prompts, JSON validation, PDF and image preprocessing Error handling and cached-mode switch Request logging disabled; .env handling; README setup section

Both: README, demo script and recording, end-to-end testing, final merge, and submission on Hackathon Hub. Only one person submits, but both check the links.

8. Git workflow

- Create the repository at the start of the build window and note the creation time in the README.
- Branches: **main** (submission-ready only), **dev** (daily integration), **feature/frontend** and **feature/backend**.
- Merge into dev at least every hour. Never push to main until the testing checklist passes.
- Commit messages should describe the change, for example 'Add WhatsApp video call guide steps'. Avoid one final large commit.
- Check before every push: no .env file, no API key, no node_modules committed.

9. Timeline (4.5 hours of build, 30 minutes of buffer)

Time (from 9:00 AM)	Person A (frontend)	Person B (backend)	Both / checkpoint
0:00 to 0:20	Vite app, routing, Home screen	Express server, routes stubbed	Agree API contract; create repo; Commit 1
0:20 to 0:50	Settings (text size, contrast), Privacy screen	Sample bills JSON, glossary JSON, cached results	Merge to dev; Commit 2
0:50 to 1:30	Bill upload UI, result cards with masked numbers, glossary sheet	/api/extract-bill in cached mode, then sample endpoint	Test F1 and F2 with samples; Commit 3
1:30 to 2:10	Guide list and step view; WhatsApp guides UI	WhatsApp guide JSON content (4 guides)	SCOPE LOCK at 1:30. Commit 4
2:10 to 2:40	Gmail guide with mock screens	Screen helper cache and /api/explain-screen (cached)	Review mock screen labels; Commit 5
2:40 to 3:10	Urdu labels, polish, responsive check	Live mode for bill extraction (behind a flag)	Commit 6
3:10 to 3:30	Bug fixes from checklist	Live mode stability test; error states	Freeze live mode if not stable; Commit 7
3:30 to 3:50	Final UI pass, no new features	Final cache regeneration; verify cached mode	Full testing checklist (Section 10)
3:50 to 4:10	Demo recording (cached mode)	Demo recording support, screen capture	Video under 2 minutes; check the link
4:10 to 4:25	README sections for UI and guides	README sections for setup, AI, data	Final README; final commit ID
4:25 to 4:30	-	-	Submit on Hackathon Hub. Hard stop.

Why this is low risk: the demo runs in cached mode, so network, rate limits and API outages cannot break the judged flow. Live mode is an added bonus that is only shown if it is stable. The 30-minute buffer and the 4:30 PM submission target cover upload problems.

10. Testing checklist (run at 3:30 PM and again at 4:10 PM)

- [ ] All three sample bills show correct cards and the AI estimate note.
- [ ] Account numbers show only the last 4 digits everywhere in the UI.
- [ ] A blank, wrong-type or oversized file shows a friendly error and a retry button, not a crash.
- [ ] Every guide: Next and Back work on every step; 'Step X of Y' is correct; first and last steps behave.
- [ ] Gmail guide works end to end without a network connection.

- [ ] Glossary bottom sheet opens and closes on a phone-sized screen.
- [ ] Text size and contrast toggles apply to every screen and survive a page reload.
- [ ] Privacy screen is reachable from Home in two taps or fewer.
- [ ] Cached mode works with the network turned off.
- [ ] Urdu labels appear where translated; no broken characters.
- [ ] Repository search finds no API key, .env file or node_modules folder.
- [ ] Repository and demo video links open in a private browser window.

11. Demo script (under 2 minutes, cached mode)

Time	What is shown	What is said
0:00 to 0:15	Home screen	A parent gets stuck on a bill and on WhatsApp video calls. Agahi teaches them step by step.
0:15 to 0:50	Sample bill upload, result cards, one glossary term, AI estimate note	This is a sample bill, so no personal data is used. Account number is masked. Check the original bill.
0:50 to 1:15	WhatsApp video call guide, 3 steps	Each step has one action. The screens are recreated mock screens, not the real app.
1:15 to 1:25	Gmail attachment guide, 2 steps	Mock screens; no Gmail login needed.
1:25 to 1:40	Privacy screen	What is stored (nothing for uploads), what is not, and that AI can make mistakes.
1:40 to 1:55	Settings: text size and contrast	Accessibility for older eyes.
1:55 to 2:00	Title card or README	Clearly state what is live, what is cached, and what is mocked.

12. Submission checklist

- README includes: project title and team members; target user and problem; solution and core features; setup and run steps; technology stack; AI tools used and their role (for example, 'Claude used for code suggestions and copy drafting'); third-party credits (fonts, icons, libraries); known limitations and mocked features; final submitted commit ID; note that work started on the build day.
- At least 8 commits spread across the build window, visible in history.
- Demo video under 2 minutes, with an accessible link. Clearly separate working features from simulations.
- Optional live link works; if included, test-account instructions have no personal credentials.
- Submitted through Hackathon Hub, not only shared elsewhere.
- Both teammates available at the assigned live judging time, able to explain implementation choices.

13. If you fall behind: cut order

Step	Cut	Effect on score
1	Live bill extraction (F9); ship cached mode	Minimal; say so honestly
2	Screen helper (F7)	Minor; Gmail and WhatsApp guides still cover the brief
3	Urdu labels beyond Home and guide titles	Minor to Design score

Step	Cut	Effect on score
4	Text size and contrast toggles (keep large default buttons)	Minor to Design score
Never	Privacy screen, AI estimate note, masked numbers, mock disclaimers, cached mode	These support Trust, Safety and Privacy scoring

14. Rubric alignment

Criterion (weight)	How Agahi meets it
Ideation and Originality (20%)	Combines bill explanation with teaching everyday digital tasks, rather than just doing them for the user
Target User Understanding (16%)	Named persona: a parent or grandparent, 55 to 75; real pain points from bills, WhatsApp and email
Design and UI (16%)	One step per screen, large buttons, text size and contrast controls, plain language
Trust, Safety and Privacy (16%)	Privacy screen, AI estimate warning, masked numbers, no storage, no password or OTP requests
Execution (16%)	Working guides and bill cards in cached mode, honest labelling of mocks, error handling
Impact and Presentation (16%)	Clear two-minute story, one user journey, honest explanation of what is live and what is mocked